

UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
KATEDRA ZA AUTOMATIKU I UPRAVLJANJE SISTEMIMA

Simulacija: Model opisan algebarskim jednačinama

Modeliranje i simulacija sistema

Upravljanje, modelovanje i simulacija sistema

Uvod

- Razmatramo: simulacija sistema opisanog matematičkim modelom
Zadatak: računanje izlaza modela na osnovu poznatih ulaza
- Rešenje: analitičko i/ili numeričko (računarski program)
- Tipovi (opis) modela i numeričko rešavanje:
 - Linearan sistem algebarskih jednačina
 - Nelinearan sistem algebarskih jednačina
 - Sistem običnih diferencijalnih jednačina (nelinearan)
 - Sistem linearnih običnih diferencijalnih jednačina

Numeričko rešavanje sistema algebarskih jednačina

- Numeričko rešavanje sistema **linearnih** algebarskih jednačina
 - Regularan slučaj
 - Preodređen sistem jednačina – metoda najmanjih kvadrata
- Numeričko rešavanje sistema **nelinearnih** algebarskih jednačina

Sistem linearnih algebarskih jednačina

- Razvijeni oblik:

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m$$

Nepoznate su $x_1, x_2, \dots, x_n$ dok su svi a i b koeficijenti (parametri modela) poznati.

U opštem slučaju broj jednačina m nije jednak broju promenljivih n .

- Matrična notacija:

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \cdots \\ b_m \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}$$

Matematičko rešenje: $\mathbf{x} = \mathbf{A}^{-1} \cdot \mathbf{b}$

Regularan sistem linearnih algebarskih jednačina

- Regularan sistem ima jednak broj nepoznatih i jednačina, tj. $n == m$
- Uslog za postojanje rešenja je $\det(A) \neq 0$

- Primer:

$$\begin{aligned}x_1 + 2x_2 - x_3 &= -2.5 \\ -x_1 + 3x_2 + 2x_3 &= 0.5 \\ -x_1 - x_2 + x_3 &= 1.5\end{aligned}$$

- Analitičko rešenje:

$$\begin{aligned}x_1 + 2x_2 - x_3 &= -2.5 \\ -x_1 + 3x_2 + 2x_3 &= 0.5 \\ -x_1 - x_2 + x_3 &= 1.5\end{aligned}$$

$L_2 \leftarrow L_2 + L_1$
 $L_3 \leftarrow L_3 + L_1$

$$\begin{aligned}x_1 + 2x_2 - x_3 &= -2.5 \\ 5x_2 + x_3 &= -2 \\ x_2 &= -1\end{aligned}$$

Imamo rešenje, ali formalno treba da nastavimo

$L_3 \leftarrow L_3 - \frac{1}{5}L_2$

$$\begin{aligned}x_1 + 2x_2 - x_3 &= -2.5 \\ 5x_2 + x_3 &= -2 \\ -\frac{1}{5}x_3 &= -\frac{3}{5}\end{aligned}$$

$x_3 = 3$
 $x_2 = -1$
 $x_1 = 2.5$

Numeričko (Julija) rešenje

```
julia> A = [1 2 -1; -1 3 2; -1 -1 1.]
3×3 Array{Float64,2}:
 1.0  2.0 -1.0
-1.0  3.0  2.0
-1.0 -1.0  1.0

julia> b = [-2.5; 0.5; 1.5]
3-element Array{Float64,1}:
-2.5
 0.5
 1.5

julia> x = gausovaEliminacija(A,b)
3-element Array{Float64,1}:
 2.5
-1.0
 2.9999999999999996

julia> A*x      # provera
3-element Array{Float64,1}:
-2.4999999999999996
 0.4999999999999999
 1.4999999999999996

julia> y = A \ b
3-element Array{Float64,1}:
 2.5000000000000004
-1.0
 3.0000000000000004
```

```
function gausovaEliminacija(A, b)
    A = hcat(A, b)
    brJedn = size(A, 1)

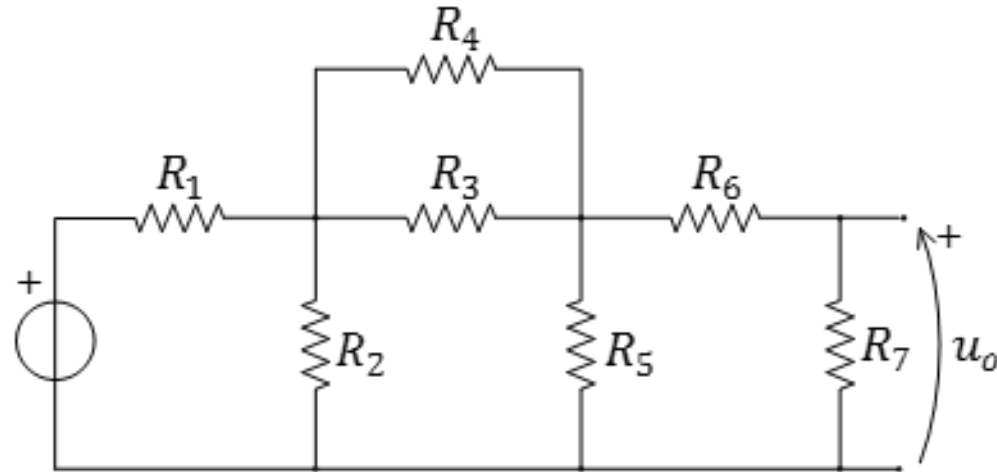
    for i in 1 : brJedn-1
        _, maxInd = findmax(abs.(A[i:end,i]))
        if maxInd != 1
            maxInd += i-1
            A[i,:], A[maxInd,:] = A[maxInd,:], A[i,:]
        end

        pivot = A[i,i]
        for j = i+1 : brJedn
            baza = A[j,i] / pivot
            A[j,:] = A[j,:] - (baza .* A[i,:])
        end
    end

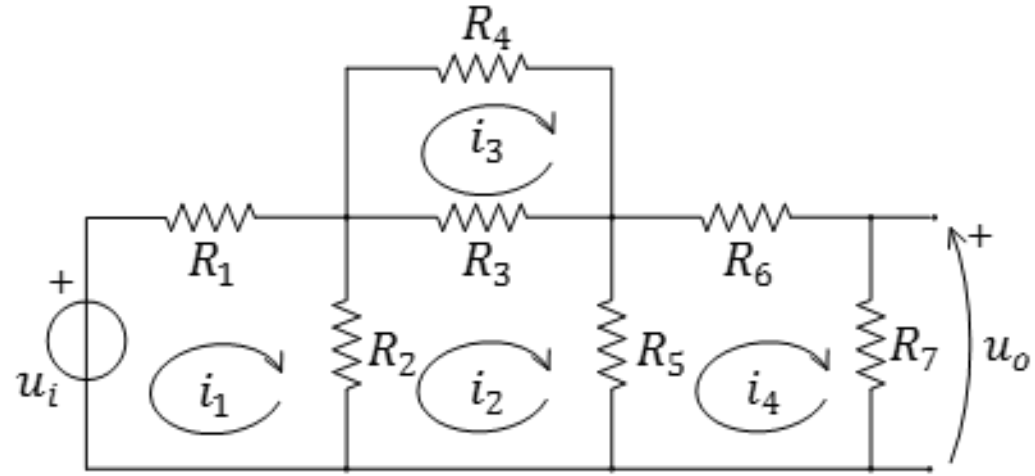
    x = A[:, end]
    x[end] /= A[end,end-1]
    for i in brJedn-1 : -1 : 1
        pivot = A[i,i]
        x[i] -= sum(A[i,i+1:end-1] .* x[i+1:end])
        x[i] /= pivot
    end
    return x
end
```

Primer 1

Upotrebom softvera za električno kolo sa slike odrediti napon na izlazu u_o kada je ulazni napon $u_i=12\text{V}$. Otpornosti otpornika su $R_1=0.5\Omega$, $R_2=5\Omega$, $R_3=3\Omega$, $R_4=6\Omega$, $R_5=4\Omega$, $R_6=10\Omega$, $R_7=2\Omega$.



Rešenje



$$\begin{aligned}(R_1 + R_2)i_1 - R_2i_2 &= u_i \\ -R_2i_1 + (R_2 + R_3 + R_5)i_2 - R_3i_3 - R_5i_4 &= 0 \\ -R_3i_2 + (R_3 + R_4)i_3 &= 0 \\ -R_5i_2 + (R_5 + R_6 + R_7)i_4 &= 0\end{aligned}$$

$$\begin{bmatrix} R_1 + R_2 & -R_2 & 0 & 0 \\ -R_2 & R_2 + R_3 + R_5 & -R_3 & -R_5 \\ 0 & -R_3 & R_3 + R_4 & 0 \\ 0 & -R_5 & 0 & R_5 + R_6 + R_7 \end{bmatrix} \cdot \begin{bmatrix} i_1 \\ i_2 \\ i_3 \\ i_4 \end{bmatrix} = \begin{bmatrix} u_i \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} R_1 + R_2 & -R_2 & 0 & 0 \\ -R_2 & R_2 + R_3 + R_5 & -R_3 & -R_5 \\ 0 & -R_3 & R_3 + R_4 & 0 \\ 0 & -R_5 & 0 & R_5 + R_6 + R_7 \end{bmatrix} \cdot \begin{bmatrix} i_1 \\ i_2 \\ i_3 \\ i_4 \end{bmatrix} = \begin{bmatrix} u_i \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$U_i = 12$

$R_1=0.5; R_2=5; R_3=3; R_4=6; R_5=4; R_6=10; R_7=2;$

$R = \begin{bmatrix} R_1+R_2 & -R_2 & 0 & 0 \\ -R_2 & R_2+R_3+R_5 & -R_3 & -R_5 \\ 0 & -R_3 & R_3+R_4 & 0 \\ 0 & -R_5 & 0 & R_5+R_6+R_7 \end{bmatrix}$

$U = [U_i; 0; 0; 0]$

$Ir = R \setminus U$ # rešavanje sistema jednačina $R \cdot Ir = U$

4-element Array{Float64,1}:

4.0

2.00000000000000004

0.6666666666666666

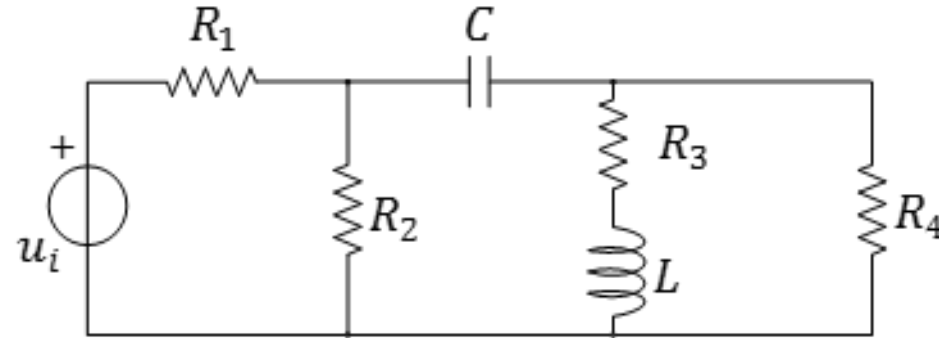
0.50000000000000001

$U_o = Ir[4] * R_7$ # izlazni napon

1.00000000000000002

Primer 2

Neka je naponski izvor sinusnog talasnog oblika učestanosti 1000 rad/s i amplitude $10V$. Odrediti amplitude i fazne pomake napona na otporniku R_4 , kalemu i kondenzatoru. Poznato je: $R_1 = 1k\Omega$, $R_2 = 500\Omega$, $R_3 = 200\Omega$, $R_4 = 500\Omega$, $C = 2\mu F$, $L = 0.5mH$.



```

R1=1000; R2=500; C=2e-6; L=0.5e-3; R3=200; R4=500;
Ui = 10; w = 1e4;                                # Ui(t)=Ui*sin(w*t)
Z = [ R1+R2    -R2                                0
      -R2      R2+R3+im*w*L+1/(im*w*C)  -R3-im*w*L
      0        -R3-im*w*L                R3+R4+im*w*L ];
E = [Ui; 0; 0];
Ic = Z\E                                           # konturne struje

```

```

3-element Array{Complex{Float64},1}:
 0.008976974005608224 + 0.00023019744743318576im
 0.006930922016824672 + 0.0006905923422995574im
 0.0019769927545535957 + 0.00023269730681609548im

```

```

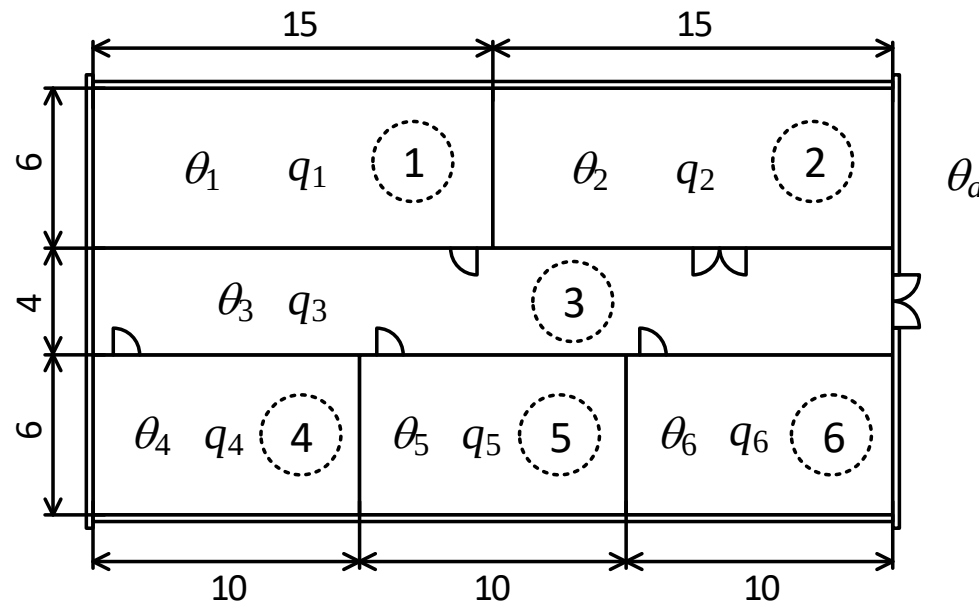
U4 = R4 * Ic[3];                                # napon na otporniku R4
UL = im*w*L*(Ic[2]-Ic[3]);                       # napon na kalemu
UC = 1/(im*w*C)*Ic[2];                          # napon na kondenzatoru

U = [U4 UL UC]                                   # ispis napona u vektoru
Um = abs.(U)                                     # amplitude-moduli napona
Ufi = angle.(U)*180/pi                          # fazni pomaci u stepenima

```

Primer 3 Grejanje zgrade

- Napisati programski kod za izračunavanje potrebne snage grejača da bi u prostorijama bila temperatura od 25 °C dok je spoljašnja temperatura 0 °C

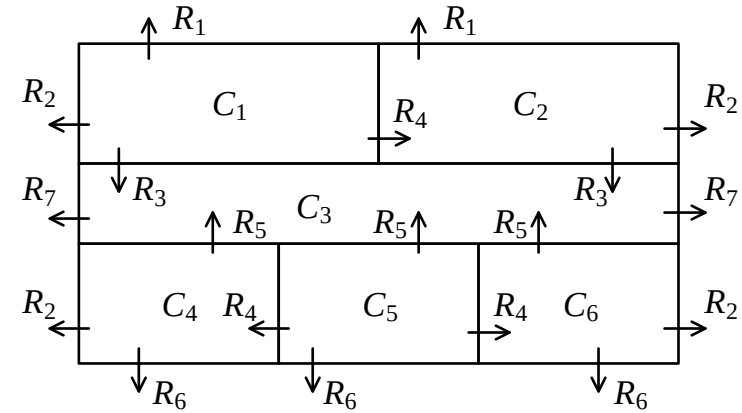


Matematički model

$$\underline{A\bar{\theta} + \bar{Q} + b\theta_a = 0}$$

$$\bar{\theta} = \begin{bmatrix} \bar{\theta}_1 \\ \bar{\theta}_2 \\ \bar{\theta}_3 \\ \bar{\theta}_4 \\ \bar{\theta}_5 \\ \bar{\theta}_6 \end{bmatrix}$$

$$\bar{Q} = \begin{bmatrix} \bar{q}_1 \\ \bar{q}_2 \\ \bar{q}_3 \\ \bar{q}_4 \\ \bar{q}_5 \\ \bar{q}_6 \end{bmatrix}$$



$$A = \begin{bmatrix} a_1 & \frac{1}{R_4} & \frac{1}{R_3} & 0 & 0 & 0 \\ \frac{1}{R_4} & a_2 & \frac{1}{R_3} & 0 & 0 & 0 \\ \frac{1}{R_3} & \frac{1}{R_3} & a_3 & \frac{1}{R_5} & \frac{1}{R_5} & \frac{1}{R_5} \\ 0 & 0 & \frac{1}{R_5} & a_4 & \frac{1}{R_4} & 0 \\ 0 & 0 & \frac{1}{R_5} & \frac{1}{R_4} & a_5 & \frac{1}{R_4} \\ 0 & 0 & \frac{1}{R_5} & 0 & \frac{1}{R_4} & a_6 \end{bmatrix}$$

$$a_1 = -\left(\frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3} + \frac{1}{R_4}\right)$$

$$a_2 = -\left(\frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3} + \frac{1}{R_4}\right)$$

$$a_3 = -\left(\frac{2}{R_3} + \frac{3}{R_5} + \frac{2}{R_7}\right)$$

$$a_4 = -\left(\frac{1}{R_2} + \frac{1}{R_4} + \frac{1}{R_5} + \frac{1}{R_6}\right)$$

$$a_5 = -\left(\frac{2}{R_4} + \frac{1}{R_5} + \frac{1}{R_6}\right)$$

$$a_6 = -\left(\frac{1}{R_2} + \frac{1}{R_4} + \frac{1}{R_5} + \frac{1}{R_6}\right)$$

$$b = \begin{bmatrix} \frac{1}{R_1} + \frac{1}{R_2} \\ \frac{1}{R_1} + \frac{1}{R_2} \\ \frac{2}{R_7} \\ \frac{1}{R_2} + \frac{1}{R_6} \\ \frac{1}{R_6} \\ \frac{1}{R_2} + \frac{1}{R_6} \end{bmatrix}$$

```

# betonski zid (spoljašnji)
Fb = 0.5/1.55;
# betonski zid-tanak (unutrašnji)
Fbt = 0.2/1.55;
# prozor: 2 x staklo 5mm + vazduh 2cm
Fp = 0.01/0.8 + 0.02/0.026;

R1b = Fb/(15 * 1.5); R1p = Fp/(15 * 1.5);
R1 = 1/(1/R1b+1/R1p); P1 = 1/R1;
R2 = Fb/(6 * 3); P2 = 1/R2;
R3 = Fbt/(15 * 3); P3 = 1/R3;
R4 = Fbt/(6 * 3); P4 = 1/R4;
R5 = Fbt/(10 * 3); P5 = 1/R5;
R6b = Fb/(10 * 1.5); R6p = Fp/(10 * 1.5);
R6 = 1/(1/R6b+1/R6p); P6 = 1/R6;
R7 = Fp/(4 * 3); P7 = 1/R7;

a1 = -(P1+P2+P3+P4);
a2 = a1;
a3 = -(2*P3+2*P7+3*P5);
a4 = -(P2+P4+P5+P6);
a5 = -(2*P4+P5+P6);
a6 = a4;

```

```

A = [ a1 P4 P3 0 0 0
      P4 a2 P3 0 0 0
      P3 P3 a3 P5 P5 P5
      0 0 P5 a4 P4 0
      0 0 P5 P4 a5 P4
      0 0 P5 0 P4 a6 ]

```

```

b = [ P1+P2
      P1+P2
      2*P7
      P2+P6
      P6
      P2+P6 ]

```

```

# temperature u prostorijama [C]
T = [25; 25; 25; 25; 25; 25]; Ta = 0;
# spoljašnja temperatura [C]
Q = -A*(T .+ 273)-b*(Ta .+ 273)

```

Metoda najmanjih kvadrata

- Uvedu se greške jednačina $e_1, e_2, \dots$:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$



$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n - b_1 = e_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n - b_2 = e_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n - b_m = e_m$$

- Ukupno odstupanje:

$$\mathcal{J} = \frac{1}{2} \sum_{k=1}^m e_k^2$$

- Problem postaje

$$\min_x \mathcal{J} = \min_x \frac{1}{2} \sum_{k=1}^m e_k^2 = \min_x \frac{1}{2} \sum_{k=1}^m (a_{k1}x_1 + a_{k2}x_2 + \dots + a_{kn}x_n - b_k)^2$$

...

- Potreban uslov za minimum $\mathcal{J} = \frac{1}{2} \sum_{k=1}^m e_k^2$ je $\nabla \mathcal{J} = \mathbf{0}$

- tj.

$$\frac{\partial \mathcal{J}}{\partial x_i} = 0, \quad i = 1, 2, \dots, n$$

$$\frac{\partial \mathcal{J}}{\partial x_i} = \frac{\partial}{\partial x_i} \left(\frac{1}{2} \sum_{k=1}^m e_k^2 \right) = \frac{\partial}{\partial x_i} \left(\frac{1}{2} \sum_{k=1}^m (a_{k1}x_1 + \dots + a_{ki}x_i + \dots + a_{kn}x_n - b_k)^2 \right) = \sum_{k=1}^m e_k \frac{\partial e_k}{\partial x_i}$$

$$\frac{\partial \mathcal{J}}{\partial x_i} = \sum_{k=1}^m a_{ki} e_k = [a_{1i} \quad a_{2i} \quad \dots \quad a_{mi}] \begin{bmatrix} e_1 \\ e_2 \\ \dots \\ e_m \end{bmatrix} = 0$$

$$\nabla \mathcal{J} = \begin{bmatrix} 0 \\ 0 \\ \dots \\ 0 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & \dots & a_{m2} \\ \dots & \dots & \dots & \dots \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ \dots \\ e_m \end{bmatrix} = \mathbf{A}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = \mathbf{0}$$

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

Pseudoinverzija matrice A:

$$\mathbf{A}^* = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$$

$$\nabla \mathcal{J} = \begin{bmatrix} \frac{\partial \mathcal{J}}{\partial x_1} \\ \frac{\partial \mathcal{J}}{\partial x_2} \\ \dots \\ \frac{\partial \mathcal{J}}{\partial x_n} \end{bmatrix}$$

Primer

$$\begin{aligned}x_1 + 2x_2 - x_3 &= -2.5 \\ -x_1 + 3x_2 + 2x_3 &= 0.5 \\ -x_1 - x_2 + x_3 &= 1.5 \\ x_1 + 2x_2 + 3x_3 &= 4\end{aligned}$$

```
A = [1 2 -1; -1 3 2; -1 -1 1; 1 2 3];
b = [-2.5; 0.5; 1.5; 4];
x = A \ b                                # x = inv(A'*A)*A'*b;
3-element Array{Float64,1}:
 0.6238317757009348
-0.7001557632398757
 1.5950155763239875

c = A*x
4-element Array{Float64,1}:
-2.371495327102804
 0.4657320872274133
 1.6713395638629285
 4.008566978193146

e = A*x - b;
J = e'*e                                # greška
0.04711838006230538

x2 = [0.62; -0.7; 1.59]                 # pretpostavljeno neko drugo rešenje
e2 = A*x2 - b;
J2 = e2'*e2                             # sada je greška, tj. J2 > J
0.047499999999999999

J2 - J > 0
true
```

Numeričko rešavanje sistema nelinearnih algebarskih jednačina

- Problem:

$$f_1(x_1, x_2, \dots, x_n) = 0$$

$$f_2(x_1, x_2, \dots, x_n) = 0$$

...

$$f_m(x_1, x_2, \dots, x_n) = 0$$

- Kraće zapisano (vektorski):

$$\mathbf{f}(\mathbf{x}) = \mathbf{0}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$$

- Postupak dolaska do rešenja je iterativan:

- pretpostavi se početno rešenje $\mathbf{x}_0$
- svaki naredni korak k (iteracija) nalazi $\Delta \mathbf{x}_k$ tako da je rešenje bliže rešenju problema, tj.

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k, \quad k = 0, 1, 2, \dots$$

Ako su greške jednačina $e_1, \dots, e_m$

$$f_1(x_1, x_2, \dots, x_n) = e_1$$

$$f_2(x_1, x_2, \dots, x_n) = e_2$$

...

$$f_m(x_1, x_2, \dots, x_n) = e_m$$

Ukupno odstupanje je

$$\mathcal{J} = \frac{1}{2} \sum_{i=1}^m e_i^2 = \frac{1}{2} \sum_{i=1}^m f_i(\mathbf{x})^2$$

A rešenje $\mathbf{x}$ daje minimum

$$\min_{\mathbf{x}} \mathcal{J}$$

Njutnov postupak (Njutn-Rapson)

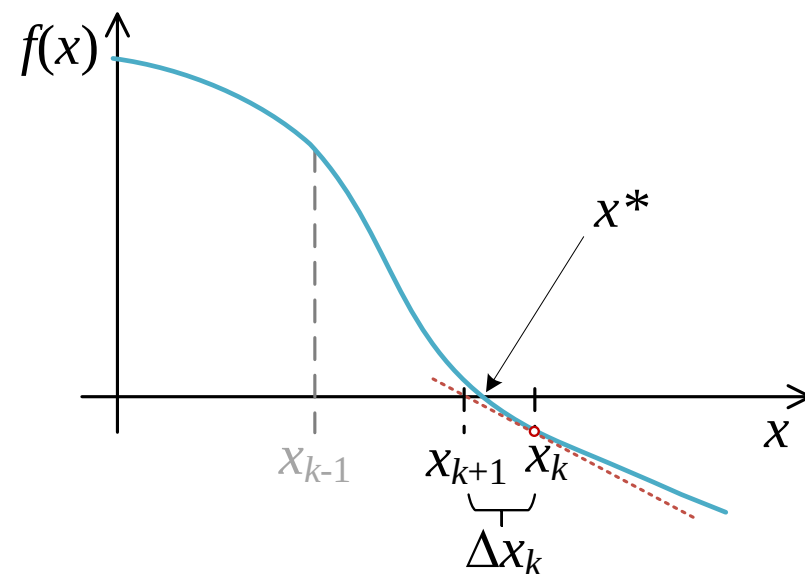
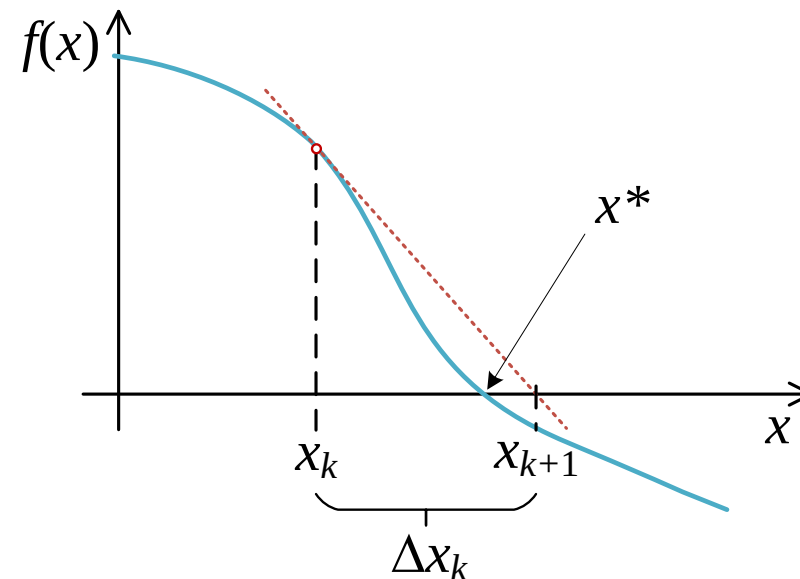
- Posmatramo jednostavniji problem (jedna promenljiva x): $f(x) = 0$
- Rešenje je iterativno:
 1. Početna procena: $x = x_0$
 2. Koriguje se procena: $x_{k+1} = x_k + \Delta x_k, k = 0, 1, \dots$
 3. Test kraja?
 1. $|\Delta x_k| \leq \varepsilon_x$ ili
 2. $\frac{1}{2} f(x_k)^2 \leq \varepsilon_f$ ili
 3. $k > k_{max}$

Nastaviti sa korakom 2.

Algoritam:

$$f(x_k + \Delta x_k) \approx f(x_k) + f'(x_k)\Delta x_k = 0$$

$$\Delta x_k = -\frac{f(x_k)}{f'(x_k)}$$



Realizacija

Uslovi kraja:

$$\begin{aligned} |\Delta x_k| &< \varepsilon_x \\ \frac{1}{2} f(x_k)^2 &< \varepsilon_J \\ k &> k_{max} \end{aligned}$$

$$\Delta x_k = -\frac{f(x_k)}{f'(x_k)}$$

```
using LinearAlgebra, # Printf

function NjutnRapson(fun, x0, epsx, epsJ, kmax)
    # Njutn-Rapson: radi za jednu promenljivu
    k = 0                    # broj iteracija
    x = x0                   # početno pogađanje
    Δx = J = Inf;
    while norm(Δx) > epsx && J > epsJ && k <= kmax
        (f, fprim) = fun(x)
        Δx = -f / fprim
        x += Δx
        J = 0.5 * f * f
        k += 1
        # @printf("%2d %35.30f\n", k, x) # kontrolni ispis
    end
    return x, J, k
end;
```

Primer

Izračunati $\sqrt{a}$, za dato a .

Rešenje:

$$x = \sqrt{a}$$

$$x^2 - a = 0$$

$$f(x) = x^2 - a$$

$$f'(x) = 2x$$

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

$$x_{k+1} = x_k - \frac{x_k^2 - a}{2x_k}$$

$$x_{k+1} = 0.5(x_k + \frac{a}{x_k})$$

```
function modelSqrt(x, a)
    f = x^2 - a
    fprim = 2x
    return f, fprim
end
```

```
u = NjutnRapson((x)->modelSqrt(x,4.0),
    1.0, eps(), eps(), 10);
```

↓
function NjutnRapson(fun, x0, epsx, epsJ, kmax)

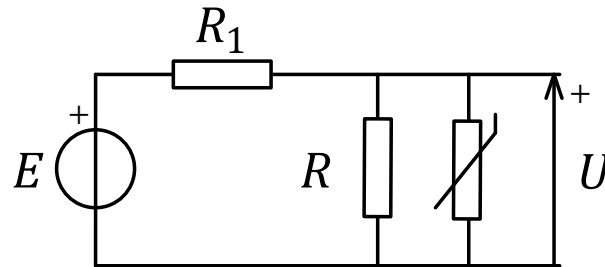
Rezultati za: $a = 4, x_0 = 1, \dots$

k	x

1	2.5000000000000000
2	2.0499999999999998
3	2.0006097560975609
4	2.0000000929222947
5	2.0000000000000002
6	2.0000000000000000

Primer

Potrošač otpornosti $R = 1000 \, \Omega$ i tiritni otpornik (varistor) čija je karakteristika $I_v = AU^a$, gde je $A = 10^{-10}$ i $a = 3.5$, vezani su paralelno. Na njih je redno povezan otpornik otpornosti $R_1 = 1000 \, \Omega$ i generator promenljive elektromotorne sile E i zanemarljive unutrašnje otpornosti. Ako se E generatora menja od $1 \, kV$ do $10 \, kV$, sa korakom od $0.5 \, kV$, izračunati promenu napona na potrošaču i prikazati je na dijagramu. Takođe, prikazati dijagram promene struje kroz tiritni otpornik u zavisnosti od E generatora.

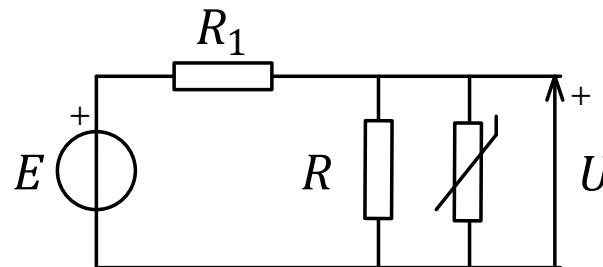


Rešenje

$$E = IR_1 + U, \quad I = U/R + AU^a$$

$$f(U) = \frac{R_1}{R}U + R_1(AU^a) + U - E = 0.$$

$$f'(U) = \frac{R_1}{R} + aR_1(AU^{a-1}) + 1.$$



```
function modelVar(U,E)
    R=1000; R1=1000; A=1e-10; a=3.5;      # parametri el. kola
    VarI(U) = A*U .^ a                    # struja varistora
    f = R1/R*U + R1*VarI(U) + U - E       # jednačina koja se rešava f(U)=0
    fprim = R1/R + a*R1*VarI(U)/U + 1    # izvod: df/dU
    return (f, fprim)
end

E = 1000:500:10000;
U = [];
for e=E
    u, _ = NjutnRapson((x) -> modelVar(x,e), 1000, 1e-3, 1e-4, 100);
    push!(U, u);
end
[E U]
```

```
19x2 Array{Any,2}:
 1000  (422.493, 9.07267e-12, 6)
 1500  (552.184, 2.87976e-5, 5)
 2000  (649.972, 5.56532e-8, 5)
 2500  (728.425, 4.69869e-11, 5)
 3000  (794.183, 1.33487e-14, 5)
 3500  (851.016, 4.8318e-6, 4)
 4000  (901.236, 9.25868e-9, 4)
 4500  (946.355, 6.85595e-13, 4)
 5000  (987.415, 7.0975e-8, 3)
 5500  (1025.16, 2.18699e-5, 3)
 6000  (1060.16, 7.14732e-12, 4)
 6500  (1092.82, 8.25689e-9, 4)
 7000  (1123.48, 8.7217e-7, 4)
 7500  (1152.39, 2.73022e-5, 4)
 8000  (1179.79, 8.55633e-16, 5)
 8500  (1205.83, 6.31176e-14, 5)
 9000  (1230.66, 2.3217e-12, 5)
 9500  (1254.42, 5.07842e-11, 5)
10000  (1277.2, 7.43099e-10, 5)
```

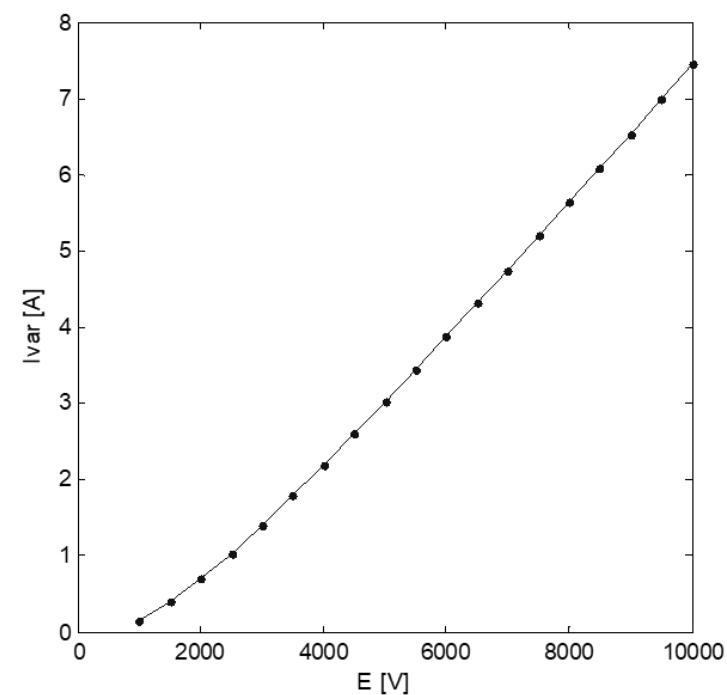
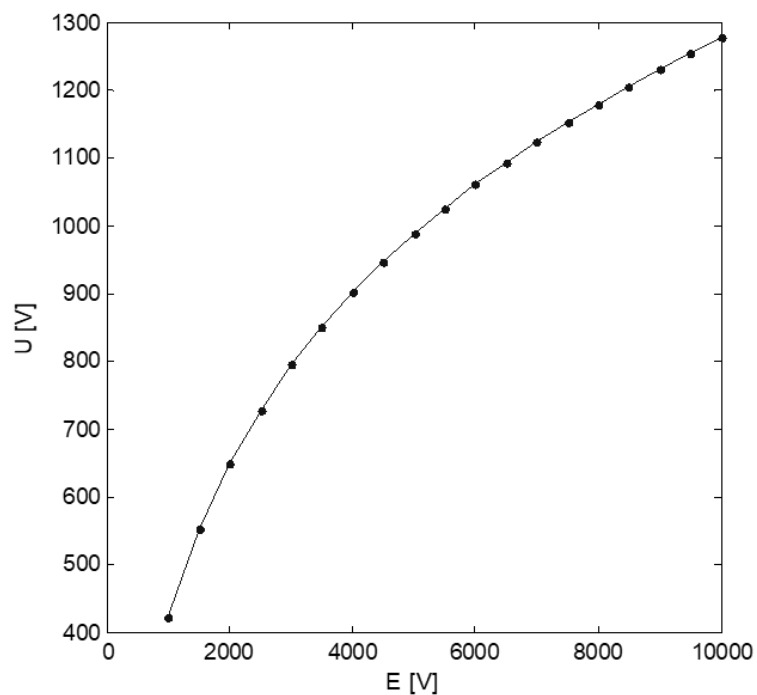
function NjutnRapson(fun, x0, epsx, epsJ, kmax)

Iteracije za $E = 1000$ V

k	J	x	f	dx
1	8662277.660168382	681.490156200	4162.277660168	-318.5098437995998
2	707127.7662891657	491.0139567617	1189.2247611694	-190.4761994387372
3	29852.51238931025	427.87268768881	244.34611676599	-63.141269072855579
4	158.0187669616667	422.5267677055768	17.7774445273592	-5.345919983230688
5	0.0060467915024	422.4932850300124	0.1099708279719	-0.033482675564278
6	0.0000000000091	422.4932837329562	0.0000042597342	-0.000001297056226731547

Rezultati

za $E = 1000$ V ... 10000 V



Gaus-Njutnov postupak

Može se primeniti da nađe $\mathbf{x}$ za koje je $\mathcal{J}(\mathbf{x})$ minimalno ($= 0$):

$$\mathcal{J}(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^m f_i^2(\mathbf{x})$$

Funkcija f_i se razvije u Tejlorov red u okolini tačke $\mathbf{x}_k$

$$f_i(\mathbf{x}_{k+1}) \approx f_i(\mathbf{x}_k) + \left. \frac{\partial f_i}{\partial x_1} \right|_{\mathbf{x}_k} \Delta x_1 + \left. \frac{\partial f_i}{\partial x_2} \right|_{\mathbf{x}_k} \Delta x_2 + \dots + \left. \frac{\partial f_i}{\partial x_n} \right|_{\mathbf{x}_k} \Delta x_n$$

Tada je

$$\mathcal{J}(\mathbf{x}_{k+1}) = \frac{1}{2} \sum_{i=1}^m f_i^2(\mathbf{x}_{k+1}) \approx \frac{1}{2} \sum_{i=1}^m \left(\frac{\partial f_i}{\partial x_1} \Delta x_1 + \frac{\partial f_i}{\partial x_2} \Delta x_2 + \dots + \frac{\partial f_i}{\partial x_n} \Delta x_n + f_i(\mathbf{x}_k) \right)^2$$

i $\Delta \mathbf{x} = [\Delta x_1 \quad \dots \quad \Delta x_n]^T$ se može odrediti metodom najmanjih kvadrata da minimizuje $\mathcal{J}(\mathbf{x}_{k+1})$.

... Kriterijum optimalnosti metode najmanjih kvadrata (podsećanje)

$$\min_x \frac{1}{2} \sum_{i=1}^m e_i^2 = \min_x \frac{1}{2} \sum_{i=1}^m (a_{i1}x_1 + a_{i2}x_2 + \dots + a_{in}x_n - b_i)^2$$

ima analogiju sa kriterijumom: $\min \mathcal{J}(\mathbf{x}_{k+1})$

$$\min_{\mathbf{x}_{k+1}} \frac{1}{2} \sum_{i=1}^m f_i^2 \approx \min_{\Delta \mathbf{x}} \frac{1}{2} \sum_{i=1}^m \left(\frac{\partial f_i}{\partial x_1} \Delta x_1 + \frac{\partial f_i}{\partial x_2} \Delta x_2 + \dots + \frac{\partial f_i}{\partial x_n} \Delta x_n + f_i(\mathbf{x}_k) \right)^2$$

$$\text{gde su: } a_{ij} \leftrightarrow \frac{\partial f_i}{\partial x_j}, \quad x_j \leftrightarrow \Delta x_j, \quad -b_i \leftrightarrow f_i(\mathbf{x}_k)$$

Stoga se rešenje za $\Delta \mathbf{x}$ može dobiti primenom metode najmanjih kvadrata.

Do minimuma $\mathcal{J}(\mathbf{x})$ se dolazi iterativno gde je $\Delta \mathbf{x}_k \equiv \Delta \mathbf{x}$, tj.

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k, \quad k = 0, 1, 2, \dots$$

... Definišemo Jakobijan J i vrednosti funkcija kao vektor f

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}, \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix}$$

Tada se kriterijum optimalnosti može napisati u vektorskom obliku

$$\mathcal{J}(\mathbf{x}_{k+1}) \approx \frac{1}{2} \left(J(\mathbf{x}_k) \cdot \Delta \mathbf{x}_k + \mathbf{f}(\mathbf{x}_k) \right)^T \cdot \left(J(\mathbf{x}_k) \cdot \Delta \mathbf{x}_k + \mathbf{f}(\mathbf{x}_k) \right)$$

gde se traži $\Delta \mathbf{x}_k$ za koje je $\mathcal{J}$ minimalno.

Iz uslova za minimum $\nabla_{\Delta \mathbf{x}} \mathcal{J} = \mathbf{0}$ (računa se gradijent $\mathcal{J}$ po $\Delta \mathbf{x}$)

$$J^T(\mathbf{x}_k) \cdot \left(J(\mathbf{x}_k) \cdot \Delta \mathbf{x}_k + \mathbf{f}(\mathbf{x}_k) \right) = \mathbf{0}$$

odakle je

$$\Delta \mathbf{x}_k = - \left(J^T(\mathbf{x}_k) \cdot J(\mathbf{x}_k) \right)^{-1} \cdot J^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

Prema analogiji sa metodom najmanjih kvadrata:

$$\mathbf{A} \leftrightarrow J(\mathbf{x}), \quad \mathbf{x} \leftrightarrow \Delta \mathbf{x}, \quad \mathbf{b} \leftrightarrow -\mathbf{f}(\mathbf{x})$$

gde je

$$\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

dobija se

$$\Delta \mathbf{x} = - \left(J^T(\mathbf{x}) J(\mathbf{x}) \right)^{-1} J^T(\mathbf{x}) \mathbf{f}(\mathbf{x})$$

Test kraja

Primetiti da se kod linearnog problema rešenje dobija u jednom koraku, dok se kod nelinearnih problema rešenje dobija kroz iteracije

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k$$

Postupak se završava kada je:

- $\|\Delta \mathbf{x}_k\| < \varepsilon_x$ ili
- $\mathcal{J}(\mathbf{x}_k) < \varepsilon_J$ ili
- $k > k_{max}$

Implementacija Gaus-Njutn algoritma

$$\Delta x = -(J^T(x)J(x))^{-1}J^T(x)f(x)$$

$$J(x) = \frac{1}{2} \sum_{i=1}^m f_i^2(x) = \frac{1}{2} f^T(x)f(x)$$

```
function GausNjutn(fun, x0; epsx=1e-6,
                    epsJ=1e-6, kmax=100, logs=false)
    k = 0                                # broj iteracija
    x = x0
    Δx = J = Inf
    stat = []
    while norm(Δx) > epsx && J > epsJ && k <= kmax
        (f, S) = fun(x)
        Δx = -(S'*S) \ (S'*f)
        x += Δx
        J = 0.5 * f' * f
        k += 1
        if logs
            push!(stat, x)
        end
    end
    return (x, J, k, stat)
end
```

Primer

$$f_1(x_1, x_2) = 2x_1 - 3x_2 - e^{-x_1} = 0$$

$$f_2(x_1, x_2) = -x_1 + 2x_2 - e^{-x_2} = 0$$

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2 + e^{-x_1} & -3 \\ -1 & 2 + e^{-x_2} \end{bmatrix}$$

```
function test(x)
    x1, x2 = x
    F = [2*x1-3*x2-exp(-x1)
         -x1+2*x2-exp(-x2)]
    J = [2+exp(-x1)  -3
         -1          2+exp(-x2)]
    return F, J
end
```

```
x, J, k = GausNjutn(test, [0.0; 0.0], logs=true);
1 1.0000 [ 1.0000 0.6667 ]
2 0.0839 [ 1.4963 0.9358 ]
3 0.0009 [ 1.5582 0.9688 ]
4 0.0000 [ 1.5589 0.9692 ]
```

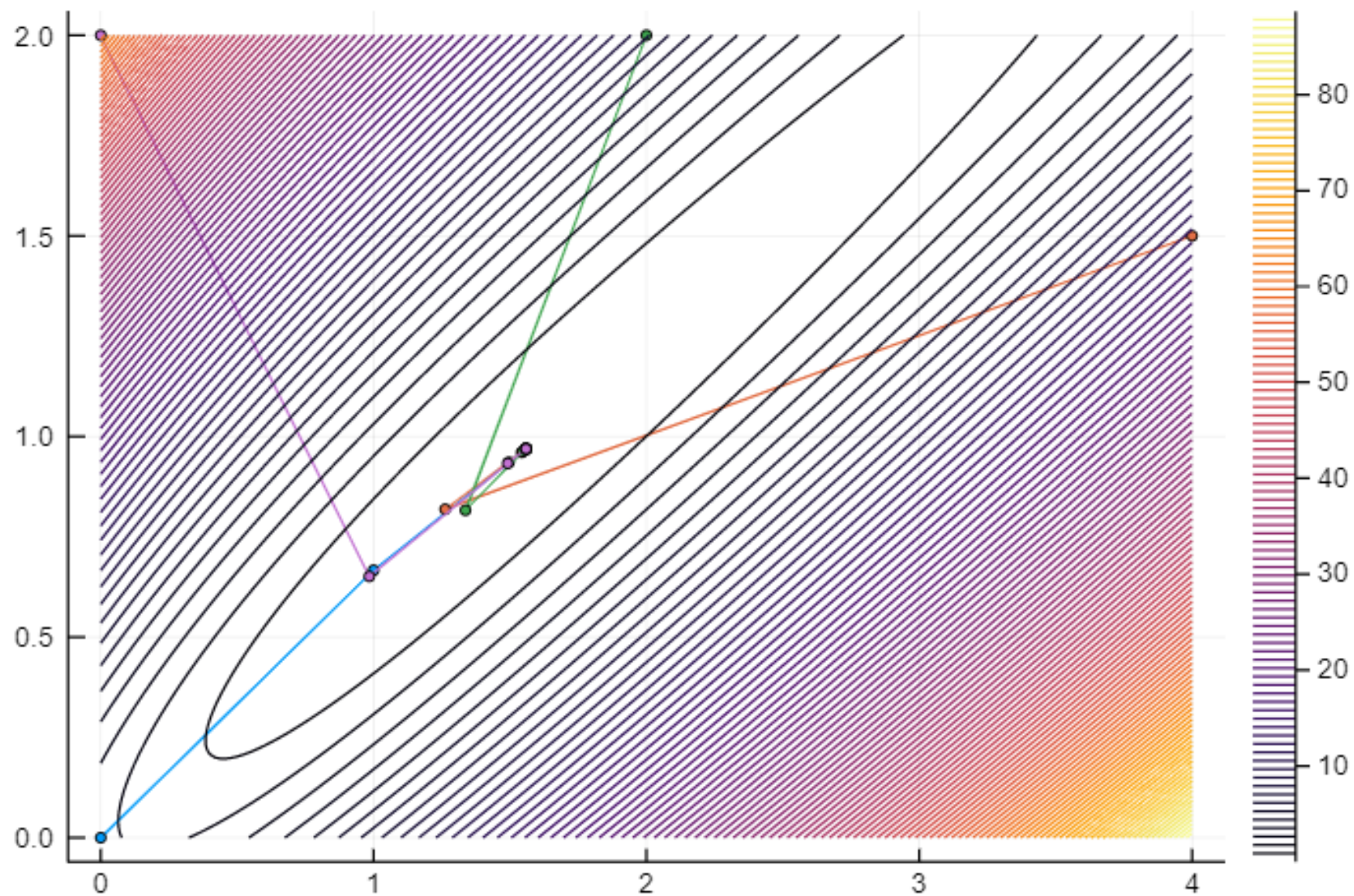
Ispis k, J, x kroz iteracije



...

GausNjutnov algoritam

Rešenje tokom iteracija polazeći iz raznih počenih pogađanja.



Gradijentni algoritam

Ideja: traži minimum od $\mathcal{J}$

$$\mathcal{J}(\mathbf{x}_0) > \mathcal{J}(\mathbf{x}_1) > \dots > \mathcal{J}(\mathbf{x}_k) > \mathcal{J}(\mathbf{x}_{k+1}) > \dots$$

gde očekujemo da je $\mathcal{J}$ smanjuje tokom iteracija.

$$\mathcal{J}(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^m f_i^2(\mathbf{x})$$

Ako je

$$\mathcal{J}(\mathbf{x}_{k+1}) = \mathcal{J}(\mathbf{x}_k + \Delta \mathbf{x}_k) \approx \mathcal{J}(\mathbf{x}_k) + \nabla^T \mathcal{J}(\mathbf{x}_k) \cdot \Delta \mathbf{x}_k$$

treba izabrati $\Delta \mathbf{x}_k$ da bude

$$\mathcal{J}(\mathbf{x}_k) > \mathcal{J}(\mathbf{x}_{k+1})$$

To se postiže za $\nabla^T \mathcal{J}(\mathbf{x}_k) \cdot \Delta \mathbf{x}_k < 0$ i biće tako kada je

$$\underline{\Delta \mathbf{x}_k = -h \cdot \nabla \mathcal{J}(\mathbf{x}_k), \quad h > 0}$$

jer je $\nabla^T \mathcal{J}(\mathbf{x}_k) \cdot \nabla \mathcal{J}(\mathbf{x}_k) > 0$ (što je suma kvadrata).

Gradijentni algoritam se naziva i **algoritam najstrmijeg pada** jer se promena $\mathbf{x}_k$ vrši suprotno od gradijenta $\nabla \mathcal{J}(\mathbf{x}_k)$ u toj tački $\mathbf{x}_k$. Inače, gradijent određuje najstrmiji porast.

Primena gradijentnog algoritma na rešavanje skupa nelinearnih jednačina

Problem koji rešavamo je $\min_x \mathcal{J}(\mathbf{x})$ gde je

$$\mathcal{J}(\mathbf{x}_k) = \frac{1}{2} \sum_{i=1}^m f_i^2(\mathbf{x}_k) = \frac{1}{2} \mathbf{f}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

Prema gradijentnom algoritmu

$$\Delta \mathbf{x}_k = -h \cdot \nabla \mathcal{J}(\mathbf{x}_k), \quad h > 0$$

Treba nam

$$\nabla \mathcal{J}(\mathbf{x}_k) = \nabla (\mathbf{f}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)) = \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

pa je

$$\Delta \mathbf{x}_k = -h \cdot \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k), \quad h > 0$$

Implementacija algoritma najstrmijeg pada

```
function NajstrmijiPad(fun, x0, h; epsx=1e-6,  
                        epsJ=1e-6, kmax=100, logs=false)  
    k = 0 # broj iteracija  
    x = x0  
    Δx = J = Inf  
    stat = []  
    while norm(Δx) > epsx && J > epsJ && k <= kmax  
        (f, S) = fun(x)  
        Δx = -h*(S'*f)  
        x += Δx  
        J = 0.5 * f' * f  
        k += 1  
        if logs  
            push!(stat, x)  
        end  
    end  
    return (x, J, k, stat)  
end
```

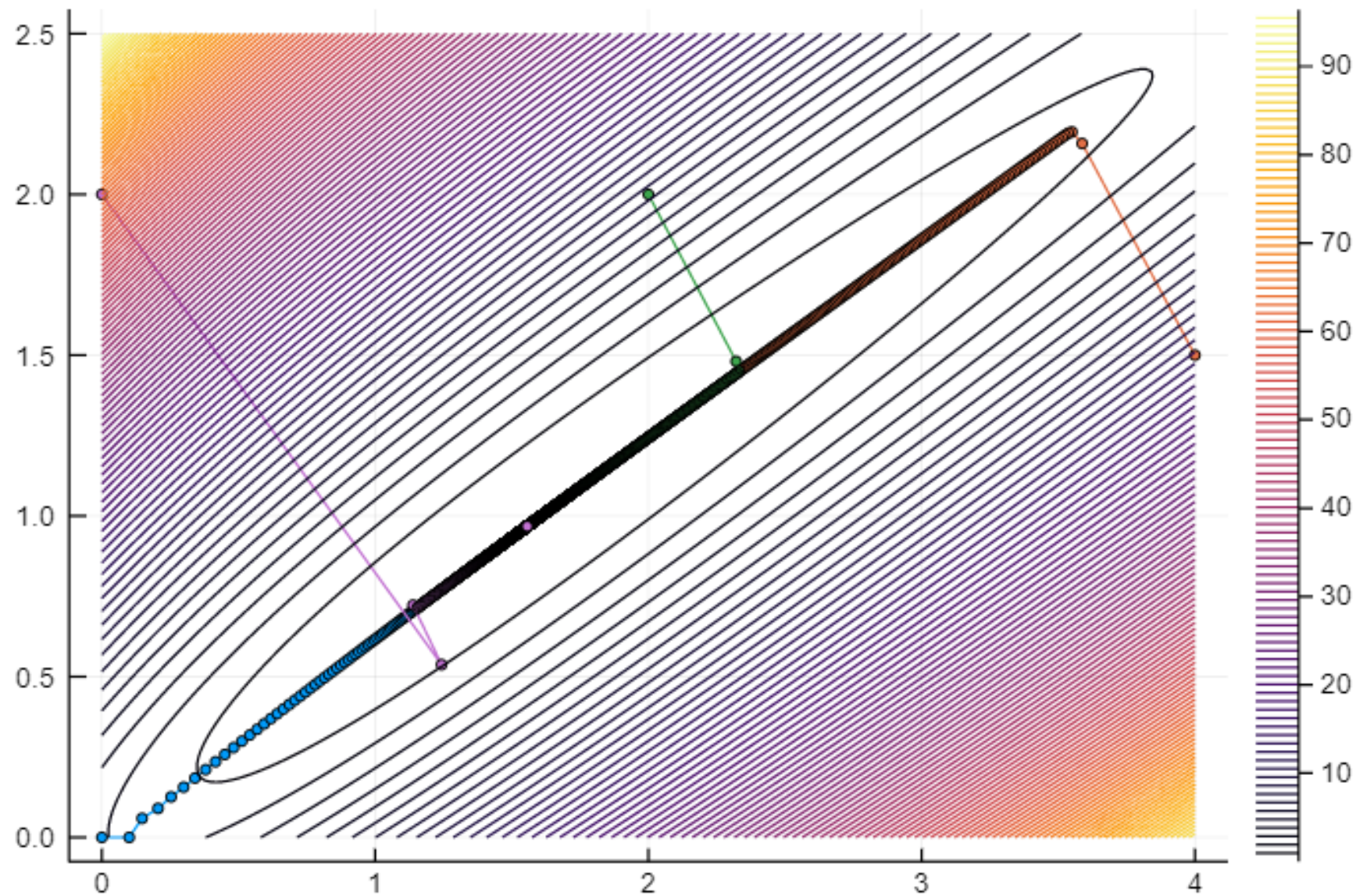
$$\Delta x = -h \cdot J^T(x) \cdot f(x)$$

$$J(x) = \frac{1}{2} \sum_{i=1}^m f_i^2(x) = \frac{1}{2} f^T(x) f(x)$$

...

Gradijentni algoritam

Rešenje tokom iteracija polazeći iz raznih počenih pogađanja.



Levenberg–Markartov algoritam

Kombinuje dobre strane gradijentnog i Gaus-Njutnovog algoritma

$$\Delta \mathbf{x}_k = -(\mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{J}(\mathbf{x}_k) + \lambda_k \mathbf{I})^{-1} \cdot \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k), \quad \lambda_k > 0$$

Adaptira λ_k tokom iteracija

$$\lambda_{k+1} = \begin{cases} \nu \lambda_k, & \mathcal{J}(\mathbf{x}_k) > \mathcal{J}(\mathbf{x}_{k-1}) \\ \frac{1}{\nu} \lambda_k, & \mathcal{J}(\mathbf{x}_k) \leq \mathcal{J}(\mathbf{x}_{k-1}) \end{cases}, \quad \nu > 1$$

$$\lambda_k \rightarrow \infty: \Delta \mathbf{x}_k = -\frac{1}{\lambda_k} \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

$$\lambda_k \rightarrow 0: \Delta \mathbf{x}_k = -(\mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{J}(\mathbf{x}_k))^{-1} \cdot \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

Gaus-Njutnov algoritam

$$\Delta \mathbf{x}_k = -(\mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{J}(\mathbf{x}_k))^{-1} \cdot \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k)$$

Gradijentni algoritam

$$\Delta \mathbf{x}_k = -h \cdot \mathbf{J}^T(\mathbf{x}_k) \cdot \mathbf{f}(\mathbf{x}_k), \quad h > 0$$

Implementacija Levenberg–Markart algoritma

$$\Delta x = -(J^T(x) \cdot J(x) + \lambda I)^{-1} \cdot J^T(x) \cdot f(x)$$

$$\lambda_{k+1} = \begin{cases} v\lambda_k, & \mathcal{J}(x_k) > \mathcal{J}(x_{k-1}) \\ \frac{1}{v}\lambda_k, & \mathcal{J}(x_k) \leq \mathcal{J}(x_{k-1}) \end{cases}$$

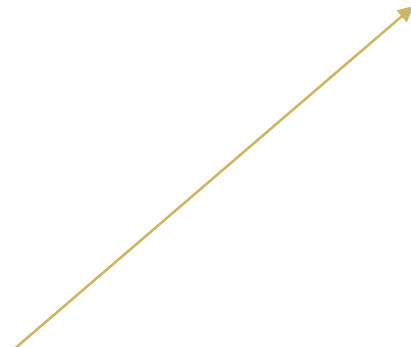
$$v > 1$$

```
function LevenbergMarkart(fun, x0; h=0.5,
                        epsx=1e-6, epsJ=1e-6, kmax=100, logs=false)
    k = 0                                # broj iteracija
    x = x0
    Jx = Js = Inf                        # prethodna vrednost J
    λ = 1/h;    ni = 1.2;
    stat = []
    while norm(Δx) > epsx && Js > epsJ && k <= kmax
        (f, S) = fun(x)
        Δx = -((S'*S) + λ*I) \ S'*f
        J = 0.5 * f' * f
        k += 1
        if J > Js
            λ = λ * ni
        else
            λ = λ / ni
            x += Δx
        end
        Js = J
        if logs
            push!(stat, x)
        end
    end
    return (x, Js, k, stat)
end
```

Primena Levenberg–Markart algoritma

```
function test(x)
    x1, x2 = x
    F = [2*x1-3*x2-exp(-x1)
         -x1+2*x2-exp(-x2)]
    J = [2+exp(-x1)  -3
         -1          2+exp(-x2)]
    return F, J
end
```

Ispis: k, J, λ, x



```
x, J, k = LevenbergMarkart(test,[0;0], logs=true);
 1  1.0000  1.6667  [ 0.4167 0.2500 ]
 2  0.4077  1.3889  [ 0.6940 0.4234 ]
 3  0.1989  1.1574  [ 0.9001 0.5542 ]
 4  0.1032  0.9645  [ 1.0614 0.6564 ]
 5  0.0542  0.8038  [ 1.1901 0.7377 ]
 6  0.0280  0.6698  [ 1.2926 0.8023 ]
 7  0.0139  0.5582  [ 1.3732 0.8529 ]
 8  0.0065  0.4651  [ 1.4348 0.8915 ]
 9  0.0028  0.3876  [ 1.4802 0.9199 ]
10  0.0011  0.3230  [ 1.5119 0.9397 ]
11  0.0004  0.2692  [ 1.5327 0.9528 ]
12  0.0001  0.2243  [ 1.5454 0.9607 ]
13  0.0000  0.1869  [ 1.5526 0.9652 ]
14  0.0000  0.1558  [ 1.5562 0.9675 ]
15  0.0000  0.1298  [ 1.5579 0.9685 ]
16  0.0000  0.1082  [ 1.5586 0.9689 ]
x
2-element Array{Float64,1}:
 1.5585743445189082
 0.9689434286545503
J
1.8618874652788131e-7
k
16
```


...

Levenberg–Markartov algoritam

Rešenje tokom iteracija polazeći iz raznih počenih pogađanja.

